Railway Animal Detection System

Communication and Alerting Layer

BTP Major (SSE)

Mid-Semester Progress Report

Student: Apoorv Singh

Roll No.: 230021004

Department: Space Science & Engineering

Institute: IIT Indore

Supervisor: Prof. Saurabh Das

TABLE OF CONTENTS

Contents

1 Introduction and Problem Statement **1**

2 Scope of Work **1**

3 System Architecture **1**
3.1 Event Processing Pipeline 2

4 Work Completed to Date **3**
4.1 Communication and alerting pipeline (complete, tested) 3
4.2 Edge node firmware (written, not yet hardware-validated) 3
4.3 Model integration 3
4.4 Prototype Security Considerations 4

5 Validation Evidence **4**

6 Current Status **5**

7 Plan for the Remainder of the Semester **6**

8 Conclusion **6**

Railway Animal Detection System

Communication and Alerting Layer

BTP Major (SSE) — Mid-Semester Progress Report

1. Introduction and Problem Statement

Animal-train collisions along Indian Railways routes cause significant loss of wildlife, occasional derailments, and recurring operational disruption. A number of ongoing efforts focus on the detection problem — using cameras, PIR motion sensors, and geophones at trackside nodes to identify when an animal is present. Detection alone, however, does not prevent a collision: the critical remaining gap is getting that information to a train driver quickly enough, and in a form simple enough, to act on within seconds, while simultaneously logging the event for a control room to track patterns over time.

This project addresses that gap directly: a **communication and alerting layer** that takes detection data from trackside nodes, determines how urgent each detection is, and pushes that decision to a driver-facing display and a control-room dashboard in real time.

2. Scope of Work

The overall Railway Animal Detection System has three parts: (a) trackside sensing hardware (ESP32-CAM, PIR, geophone), (b) a trained detection model (YOLOv8-nano, trained separately to 81.5% mAP50 on a wildlife dataset), and **(c) the communication and alerting layer** that connects detection output to human decision-makers. This report covers part (c), which is this student's specific and complete responsibility within the project: event ingestion, severity scoring, multi-node prioritization, data persistence, and the driver/control-room interfaces.

3. System Architecture

Detection events reach the system from one of two sources: a real edge node (ESP32-CAM capturing an image on PIR trigger) or, during development and testing, a simulator that posts realistic synthetic events over HTTP. Both paths converge on a single processing pipeline before touching any downstream logic. This was a deliberate design choice so that the full system — scoring, storage, and both dashboards — could be built and demonstrated before hardware was ready, with no change in behaviour once real hardware is substituted in.

Figure 1 illustrates the overall architecture and the flow of information from the trackside edge nodes through the central communication layer to the driver and control-room interfaces.

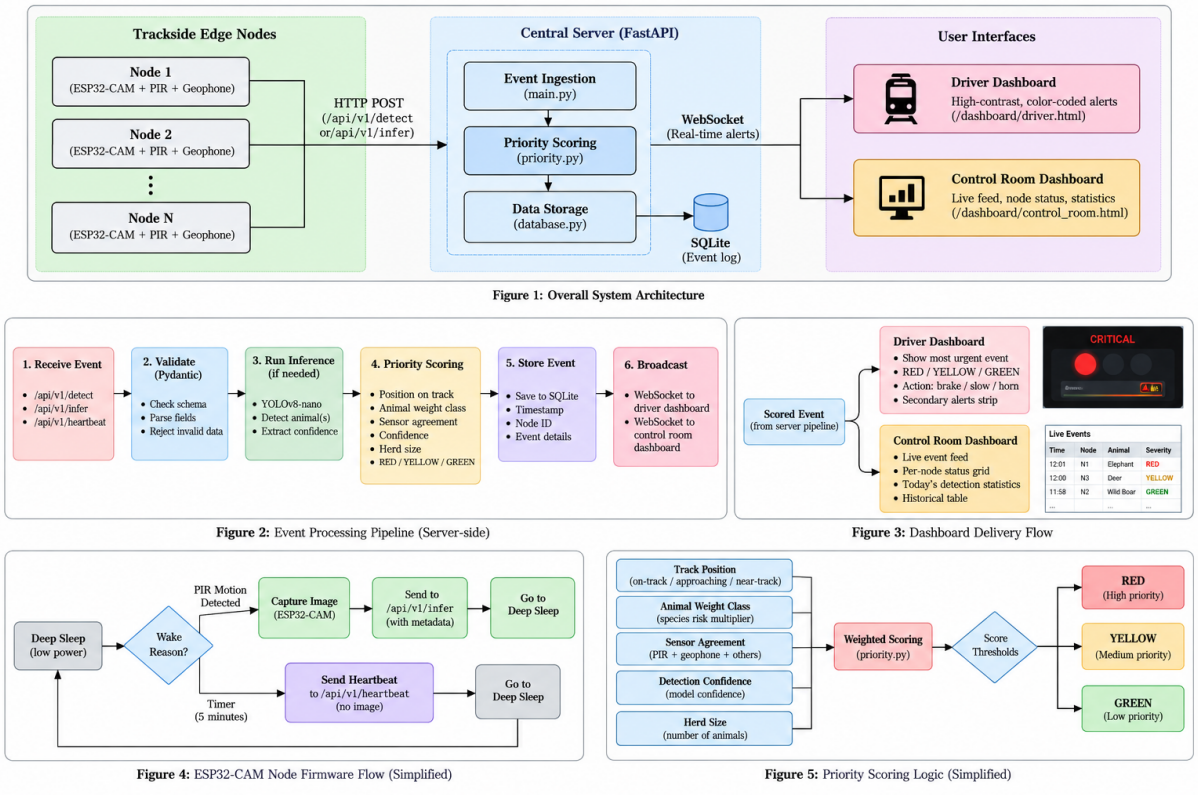


Figure 1: Overall architecture and processing flow of the railway animal detection communication and alerting layer.

3.1 Event Processing Pipeline

All detection events, irrespective of their source, are passed through a common server-side processing pipeline. Incoming requests are first received through the appropriate API endpoint and validated against the predefined Pydantic schema. When the event contains an image requiring inference, the server-side YOLOv8-nano model is invoked to obtain the detected animal class and confidence.

The validated and, where required, inferred event is then passed to the priority engine. The scoring mechanism considers track position, animal weight class, sensor agreement, detection confidence, and herd size to generate a RED, YELLOW, or GREEN severity level. The event is subsequently stored in the SQLite database and broadcast through WebSocket connections to the driver and control-room dashboards.

This common processing path ensures that events entering through simulated traffic, pre-scored detection requests, or real image-based inference receive the same downstream scoring, storage, and dashboard treatment. **The pipeline stages are:**

Ingestion : a FastAPI server exposes endpoints for pre-scored JSON events, raw images requiring inference, and lightweight heartbeats; Pydantic models validate every incoming payload against a fixed schema.

Inference (when needed) : a server-side module runs the trained YOLOv8-nano model on uploaded frames, since the ESP32-CAM lacks the compute to run inference on-device.

Priority scoring : each event is scored from track position, animal size class, multi-sensor agreement, detection confidence, and herd size, producing a RED / YELLOW / GREEN severity label.

Multi-node ranking : across simultaneously active nodes, events are ranked by severity and proximity so the driver display always shows the single most urgent situation, with others queued as secondary alerts.

Persistence : every event is logged to a database for the control room's historical record and future animal-movement path tracing.

Broadcast : scored events are pushed over WebSocket to two purpose-built dashboards: a high-contrast, color-coded driver display, and a denser control-room view with live logs and per-node status.

4. Work Completed to Date

4.1 Communication and alerting pipeline (complete, tested)

Detection event JSON schema defined as the fixed contract between edge nodes and the server.

FastAPI server with three ingestion endpoints (pre-scored JSON, raw image for inference, heartbeat) sharing one processing pipeline.

Priority/severity scoring engine with tunable, documented weights for position, animal class, sensor agreement, confidence, and herd size.

Multi-node, multi-animal prioritization and ranking logic.

SQLite-backed persistence for the control room's historical event log.

Driver dashboard: full-screen, color-coded (red/yellow/green) signal display with icon-driven action indicators (slow / brake / horn), designed for sub-second readability.

Control room dashboard: live event feed, per-node status grid, daily detection statistics, historical table.

Node simulator generating realistic multi-node detection traffic for end-to-end testing without physical hardware.

Full pipeline tested live across two physically separate devices over a real WiFi network (not local-host only), confirming the system works across an actual network boundary.

4.2 Edge node firmware (written, not yet hardware-validated)

ESP32-CAM firmware implementing deep-sleep power management with two wake sources: PIR-triggered capture-and-transmit, and a periodic timer heartbeat so node status remains meaningful between detections.

On-trigger behaviour: capture a frame, transmit it with sensor metadata to the inference endpoint over WiFi.

A flashing method identified and planned using an Arduino Mega 2560 as a USB-serial bridge, to work around a delayed component (USB programming shield).

4.3 Model integration

Server-side inference module wired to accept the trained YOLOv8-nano model and route its output through the same scoring pipeline used for all other events.

4.4 Prototype Security Considerations

The current implementation is intended for controlled prototype testing over a campus WiFi network. In particular, the API currently uses a permissive CORS configuration with `allow_origins=["*"]`. This allows requests from arbitrary origins and simplifies development and testing across multiple devices, but it should not be retained in a production railway deployment.

Security hardening, including restricted CORS(Cross Origin resource SHaring) origins, authentication, and secure communication, is therefore identified as a future development task.

5. Validation Evidence

The developed dashboards were tested using live detection events transmitted through the FastAPI communication layer. The control-room interface provides continuous visibility of node status, incoming events, detected animal information, position, severity, and priority score. The driver-facing interface converts the processed severity into a simple visual signal and corresponding driving action indicators, allowing the alert to be interpreted rapidly.

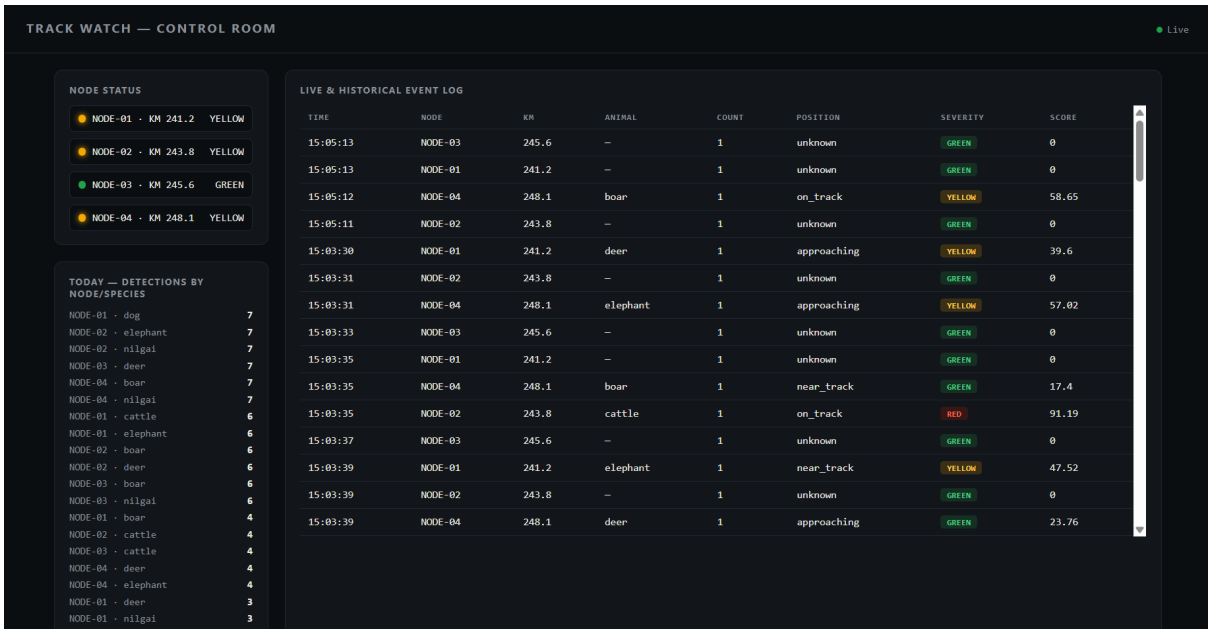


Figure 2: Live control-room dashboard showing node status, event history, detected species, position, severity, and priority scores.

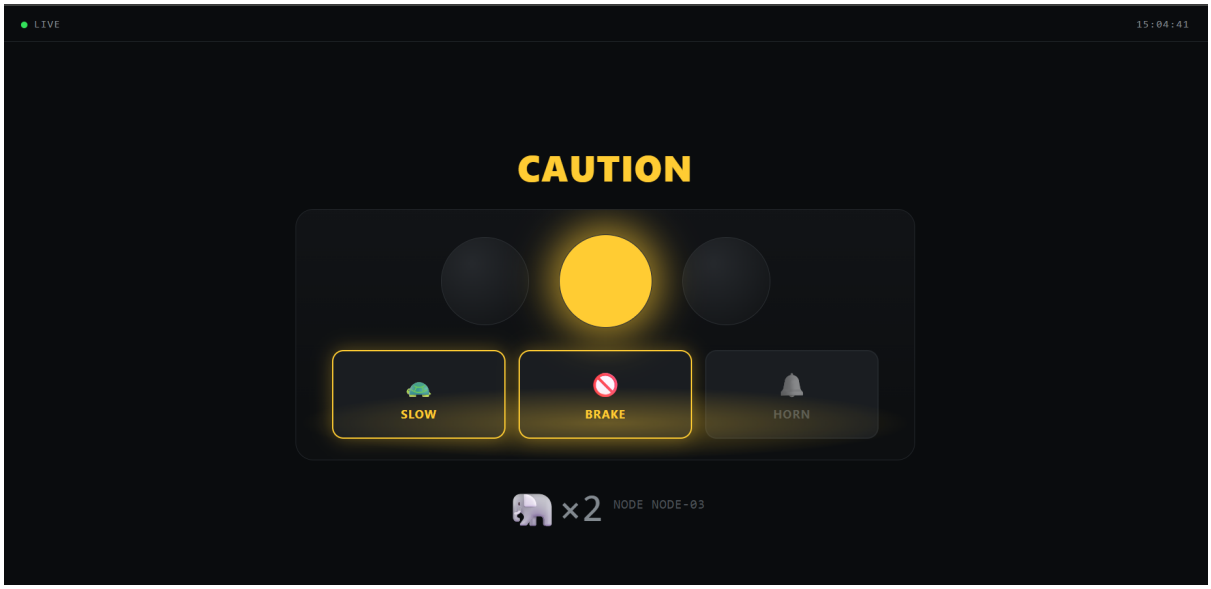


Figure 3: Live driver dashboard in CAUTION state, indicating the required slow and brake actions for an active animal detection.

The following figures demonstrate the operation of both interfaces under different event conditions. The control-room dashboard records and presents the incoming events, while the driver dashboard converts the resulting priority level into an immediately recognizable alert state.

The screenshots demonstrate the end-to-end operation of the communication and alerting layer. Detection events generated by the simulator were received by the FastAPI server, processed through the priority-scoring pipeline, stored in the event database, and subsequently reflected on the dashboards. The control-room interface provides a detailed view for monitoring multiple nodes, whereas the driver interface presents only the information required for immediate action. The RED, YELLOW, and GREEN states are generated from the corresponding priority scores, with the driver interface providing distinct visual and action indicators for each severity level.

Both states were produced by posting real detection events to the FastAPI server and observing the dashboard update live over a WebSocket connection across two physically separate devices on the same WiFi network. This confirms that the communication pipeline functions across an actual network boundary rather than only within a single-process test.

6. Current Status

The communication and alerting layer is fully functional and validated end-to-end using simulated node traffic over a real WiFi network — this is the current, reliable proof of concept. Edge node firmware is written and desk-reviewed but has not yet been flashed to physical hardware, due to a delayed USB programming shield in the ordered hardware batch; a workaround (Mega 2560 as a USB-serial bridge) has been identified and is the immediate next task. A geophone sensor, part of the original three-sensor design, is not present in the hardware procured so far and its inclusion is still to be resolved.

7. Plan for the Remainder of the Semester

The primary objective for the remainder of the semester is to transition the current software-validated prototype into a complete, campus-ready hardware prototype consisting of physical trackside nodes, the communication server, priority engine, and driver and control-room dashboards. The planned work is as follows:

- **Build and validate the physical trackside node:** Flash the ESP32-CAM firmware using the Mega-bridge method, integrate the PIR sensor, and establish reliable WiFi-based communication with the central server.
- **Complete the single-node end-to-end pipeline:** Validate the complete physical sequence of PIR detection → image capture → data transmission → server-side inference → priority scoring → dashboard alert.
- **Integrate additional sensing:** Wire and validate the geophone sensor, if available, and incorporate its output into the multi-sensor event-processing pipeline.
- **Campus-ready system demonstration:** Integrate the physical nodes, central server, driver dashboard, and control-room dashboard into a unified prototype and conduct testing under realistic campus deployment conditions, including wireless communication across the intended operating area.
- **Harden the communication layer:** Replace the current permissive CORS configuration (`allow_origins=["*"]`) with restricted authorized origins and introduce appropriate authentication and secure communication mechanisms as the prototype moves beyond controlled campus-WiFi testing.

8. Conclusion

The communication and alerting layer has been successfully developed and validated as a functional software prototype over a real WiFi network. The system integrates event ingestion, priority scoring, data persistence, multi-node ranking, and real-time driver and control-room dashboards.

The current work establishes a reliable software foundation, with the major remaining objective being its transition to a campus-ready hardware prototype. The next phase will focus on validating the ESP32-CAM nodes with physical sensors, completing the end-to-end hardware pipeline, and extending the system to multiple nodes under realistic deployment conditions.